

DOCUMENTO INTEGRAL DE DESARROLLO

Antioquia Natural

Gobernación de Antioquia — Secretaría de Ambiente

Contratista: Sebastián Guzmán Díaz · sguzmand@gmail.com · 3006552511

Versión 1 — 2026-07-14

1. Introducción

Este documento consolida la documentación técnica final de Antioquia Natural para el trámite de aval de paso a producción ante la Dirección TIC de la Gobernación de Antioquia. Complementa al Levantamiento de Requisitos de Software y Propuesta Técnica (alcance funcional) y a la Lista de Chequeo de Conformidad (verificación de lineamientos), ambos entregados por separado.

Antioquia Natural es una aplicación web mobile-first que permite consultar la biodiversidad del departamento (flora y fauna por subregión y grupo taxonómico), los recursos hídricos y los programas comunitarios Jóvenes pa' Lante y Guarda Cuencas, en español e inglés, accesible vía código QR desde cualquier dispositivo móvil.

2. Arquitectura General

2.1. Stack tecnológico

Capa	Tecnología	Versión
Frontend	HTML5 + CSS3 + JavaScript Vanilla (sin frameworks)	—
Backend	Node.js + Express	22 LTS / 4.x
Base de datos	MongoDB Atlas + Mongoose ODM	7.x
Caché	Redis + ioredis (modo degradado si no está disponible)	7.x / ^5
Logs	Winston (JSON estructurado + traceId por petición)	^3
Imágenes	sharp (conversión automática a WebP)	^0.34
Subida de archivos	multer (memoria, sin escribir temporales a disco)	^2.2
Autenticación admin	express-session + bcrypt (contraseña hasheada, nunca texto plano)	—
Documentación API	OpenAPI 3.0.3 (swagger.yaml) + swagger-ui-express en /api/docs	3.0.3
SAST	ESLint + eslint-plugin-security + Simgrep (p/nodejs, p/owasp-top-ten, reglas locales)	—
Tests	Jest + Supertest (integración contra las rutas reales)	^30

La autenticación admin migrará a Microsoft Entra ID (OAuth 2.0 + OIDC) una vez TI Gobernación provea Client ID y Tenant ID — ítem B1 del roadmap del proyecto.

2.2. Organización del código (backend)

El backend sigue una estructura de capas ligera: las rutas HTTP (routes/) validan la petición y delegan la lógica de negocio a servicios (services/), que a su vez usan los modelos de datos (models/) y utilidades transversales (utils/, middleware/). No implementa Clean Architecture ni Arquitectura Hexagonal en sentido estricto (los servicios llaman directo a Mongoose, sin interfaces de dominio), pero sí separa el "qué hace" del "cómo llega la petición" — ver detalle en el Chequeo de Lineamientos, sección 2.

```
backend/src/  index.js      Arranque de Express, sesión, /api/health  db.js
Conexión MongoDB (connCom) + Redis  routes/admin.js  Capa HTTP: valida, llama al servicio,
responde  services/      Lógica de negocio, sin dependencia de Express  fotoStorage.js
guardado/borrado de fotos en disco (WebP)  jplStats.js      agregaciones de
estadísticas JPL  publicacion.js  publicar un mes a JSON público + índice
inaturalistLookup.js  consulta a la API de iNaturalist  models/      Esquemas Mongoose
(JplPhoto, GcPhoto, CommunitySighting, Municipality)  middleware/      adminAuth (guard de
sesión), requestLogger (traceId)  utils/      logger (Winston), cache (Redis con TTLs)
config/      catalogo.js — grupos/subregiones/IUCN válidos (única fuente de verdad)
__tests__/      51 tests de integración (Jest + Supertest)
```

2.3. Módulos del frontend

Módulo	Contenido
Biodiversidad	Catálogo de especies por subregión/grupo taxonómico, mapa SVG interactivo, ficha con IUCN y distribución. Datos en JSON estático (species.json), sin API REST.
Agua	Fuentes hídricas y cuencas abastecedoras por subregión. Datos en JSON estático.
Comunidad — Jóvenes pa' Lante	Mapa de 90 municipios beneficiados, galería fotográfica mensual con filtros. Backend real (MongoDB + API admin).
Comunidad — Guarda Cuencas	Galería fotográfica mensual de cuencas y ríos. Backend real (MongoDB + API admin).

Módulo	Contenido
Comunidad — Especie del Mes	Especie insignia mensual con galería comunitaria. Contenido editorial, sin backend propio actualmente.
Panel admin (curadores)	Login con sesión, CRUD de fotos JPL/GC, autocompletar desde iNaturalist, publicar JSON mensual, estadísticas de cobertura.

Nota de alcance: solo los módulos Jóvenes pa' Lante y Guarda Cuencas tienen persistencia real en MongoDB a través de la API. Biodiversidad, Agua y Especie del Mes se sirven como JSON estático desde el frontend — la migración de Biodiversidad a base de datos real está documentada como plan en SCHEMA_DB.md, pendiente de implementar.

3. Infraestructura

3.1. Requisitos del servidor

Componente	Requisito
Sistema operativo	Ubuntu Server 24.04 LTS
Node.js	22 LTS
Proceso	PM2 (gestor de procesos, reinicio automático)
Proxy inverso	Nginx (proxy a Node.js, terminación SSL)
Certificado SSL	Let's Encrypt vía Certbot (TLS 1.2+)
Base de datos	MongoDB Atlas (cluster ya provisionado, plan M0)
Caché	Redis 7.x (256 MB, política allkeys-lru) — opcional, modo degradado si no está disponible

3.2. Variables de entorno requeridas

Variable	Descripción	Obligatoria
MONGODB_URI_COM	URI de conexión a la BD Comunidad (JPL, Guarda Cuencas)	Sí
SESSION_SECRET	Secreto de las sesiones del panel admin (mín. 32 caracteres)	Sí
ADMIN_PASSWORD_HASH	Hash bcrypt de la contraseña del panel de curadores (nunca texto plano)	Sí
REDIS_URL	URL del servidor Redis	No — modo degradado sin caché
LOG_LEVEL	Nivel de log: error, warn, info, debug	No
PORT	Puerto del servidor (por defecto 3000)	No

El archivo .env nunca se sube al repositorio (excluido en .gitignore). backend/.env.example documenta cada variable, incluyendo el comando para generar el hash de la contraseña admin.

3.3. Pasos de despliegue (resumen)

- 1. Preparar el servidor: Node.js 22, PM2, Nginx, Certbot, Redis (ver README.md para comandos completos).
- 2. Clonar el repositorio y configurar backend/.env a partir de .env.example.
- 3. npm install --omit=dev, luego iniciar con PM2 (pm2 start src/index.js --name antioquia-natural).
- 4. Configurar Nginx como proxy inverso y activar SSL con Certbot.
- 5. Verificar: pm2 status, pm2 logs, y curl al endpoint /api/health (debe responder mongodb y redis conectados).

El procedimiento completo, con cada comando, está en README.md (raíz del repositorio), sección "Despliegue en producción".

3.4. Estrategia de Entornos

Siguiendo la segregación de ambientes recomendada en la sección 6.1 de la Guía de Arquitectura y Buenas Prácticas de Desarrollo, el proyecto cuenta con los siguientes ambientes lógicos separados:

Ambiente	Estado
Desarrollo (Dev)	Local (localhost:3000), backend/.env. Único ambiente con datos reales hasta hoy (nombres de fotografías comunitarios provistos voluntariamente al participar en los programas) — no existe todavía separación de producción

Ambiente	Estado
QA / Staging	Activo desde 2026-07-16. Frontend: rama develop desplegada aparte en Netlify (branch deploy), sin afectar el sitio de producción. Backend: base de datos MongoDB separada (antioquia-biodiversa-qa, mismo cluster Atlas) poblada con datos 100% sintéticos (backend/src/scripts/seed_qa_data.js) — ningún dato real de participantes
Producción (Prod)	Pendiente de infraestructura provista por TI Gobernación (servidor Ubuntu 24.04, dominio institucional). El manual de despliegue (README.md) ya cubre el procedimiento completo

La gestión de datos de prueba sigue la anonimización obligatoria de la sección 6.2 de la Guía: el ambiente de QA no contiene ningún dato real, y el ambiente de Desarrollo no maneja datos sensibles tipo cédula o identificación (solo nombres de pila de fotógrafos que consintieron participar en los programas comunitarios).

4. Modelo de Base de Datos

MongoDB Atlas, base de datos "comunidad" (variable MONGODB_URI_COM). No hay una segunda conexión activa a una base "biodiversidad" en el backend actual — el catálogo de especies se sirve como JSON estático desde el frontend (biodiversidad/data/species.json), no desde MongoDB.

4.1. Colecciones en uso activo (accedidas por la API)

JplPhoto — fotos de Jóvenes pa' Lante

```
{  mes: String,                // 'YYYY-MM', requerido  orden: Number,  fotos: [String],
// rutas relativas al frontend (1-3 fotos)  credito: String,  municipio: String,
subregion: String,            // requerido, validado contra catálogo fijo (9 subregiones)
especieEs / especieEn / especieCientifico: String,  grupo: String,                //
requerido, validado contra catálogo fijo (9 grupos)  iucn: String,                //
validado contra LC/NT/VU/EN/CR/DD/NE, default DD  endemica: Boolean,  descripcionEs /
descripcionEn: String,  publicado: Boolean,  timestamps: true }
```

GcPhoto — fotos de Guarda Cuencas

```
{  mes: String, orden: Number,  foto: String,                // requerido  credito /
municipio: String,  subregion: String,            // requerido, validado contra catálogo
fijo  cuenca / tituloEs: String,                // requeridos  tituloEn / descripcionEs /
descripcionEn: String,  publicado: Boolean,  timestamps: true }
```

La validación de grupo/subregión/IUCN se centraliza en backend/src/config/catalogo.js — la misma fuente de verdad se usa en la ruta HTTP (rechazo con 400 y mensaje claro) y en el esquema de Mongoose (enum, como respaldo).

4.2. Colecciones definidas pero sin ruta activa

CommunitySighting y Municipality tienen esquema Mongoose definido en el código, pero ninguna ruta de la API los lee ni escribe actualmente — quedaron preparados para una futura integración de Especie del Mes con base de datos real, hoy esa sección funciona con JSON estático y contenido editorial manual.

4.3. Índices recomendados

```
JplPhoto: { mes: 1, orden: 1 } JplPhoto: { mes: 1, publicado: 1 } GcPhoto: { mes: 1, orden:
1 }
```

5. Seguridad

Verificación ejecutada el 2026-07-13/14, con los 4 pasos de SAST/calidad del pipeline de CI corridos manualmente contra el código real (el pipeline de Azure DevOps sigue inactivo, pendiente de las Service Connections de TI).

Verificación	Resultado
npm audit --audit-level=high	0 vulnerabilidades (3 corregidas esta sesión: multer, form-data, js-yaml)
ESLint + eslint-plugin-security	0 errores
Semgrep (p/nodejs + p/owasp-top-ten + reglas locales)	0 hallazgos
Autenticación admin	bcrypt (factor de costo 10), ya no compara contraseña en texto plano
Cookie de sesión	httpOnly explícito, nombre custom (no revela la librería), secure activo en producción
Credenciales en código fuente	Ninguna — todas se leen de variables de entorno

Verificación	Resultado
Licencias de dependencias	13 dependencias de producción: 11 MIT, 1 BSD-2-Clause, 1 Apache-2.0. Ninguna GPL/restrictiva
Cifrado en tránsito (TLS 1.2+)	Documentado y listo en el manual de despliegue (Nginx + Let's Encrypt/Certbot), pendiente de verificar en vivo hasta que exista un ambiente real desplegado
Cifrado en reposo	Provisto por defecto por MongoDB Atlas en todos sus clusters, incluido el plan gratuito M0 usado actualmente
MFA para administradores	Pendiente — la solución de fondo es Microsoft Entra ID (ítem B1 del roadmap), sujeta a que TI Gobernación provea Client ID y Tenant ID. Mientras tanto, acceso protegido con sesión + contraseña con hash bcrypt

Detalle completo, incluyendo los ítems parcialmente cumplidos (patrón de arquitectura del backend y cifrado en un ambiente todavía sin desplegar), en la Lista de Chequeo de Conformidad entregada por separado.

6. Calidad y Pruebas

Métrica	Valor
Tests de integración	51 (Jest + Supertest, contra las rutas HTTP reales)
Cobertura de líneas	98.62% (umbral configurado: 90%)
Cobertura de funciones	95.38% (umbral configurado: 90%)
Alcance de la cobertura medida	routes/admin.js, middleware/, services/
Reporte de tests	JUnit XML (jest-junit), listo para publicar en Azure DevOps

Los tests cubren: autenticación (login/logout/sesión), CRUD completo de fotos JPL y Guarda Cuencas (crear, editar, eliminar, listar, validaciones), autocompletar desde iNaturalist, agregaciones estadísticas, publicación de JSON mensual, y el middleware de logging.

7. Documentos Relacionados

Entregados por separado, como parte del mismo trámite de aval:

- Levantamiento de Requisitos de Software y Propuesta Técnica — alcance funcional, roles, requerimientos funcionales y no funcionales.
- Lista de Chequeo de Conformidad — verificación ítem por ítem contra la Guía de Arquitectura y Buenas Prácticas de Desarrollo de la Gobernación.
- README.md (raíz del repositorio) — manual de instalación y despliegue paso a paso, con todos los comandos.
- swagger.yaml (backend/src/) — especificación OpenAPI 3.0.3 de la API, navegable en /api/docs.